

RELAZIONE TECNICA DETTAGLIATA DI PROGETTO

Corso: Web and Multimedia Technologies (Laurea Magistrale)

Docente: Prof. Marco Porta — Università degli Studi di Pavia

Studente: Marco Santoro

Progetto: Algora Studio / Foliarium Web Platform

Anno Accademico: 2025/2026

1. INTRODUZIONE, VISION E CONTESTO APPLICATIVO

La presente relazione descrive in dettaglio la progettazione, l'architettura tecnica e le scelte metodologiche alla base dello sviluppo del sito web e della piattaforma di gestione per **Algora Studio**.

Algora Studio è una software house, fondata da **Marco Santoro**, specializzata nella realizzazione di soluzioni verticali per il dominio del patrimonio culturale e della memoria storica documentale italiana (Archivi di Stato, archivi storici comunali, studi notarili e fondazioni).

Il prodotto di punta attorno a cui ruota la comunicazione e l'interazione del sito è **Foliarium**, un software gestionale sviluppato per l'Archivio di Stato di Savona per la digitalizzazione, la modellazione relazionale e la consultazione *fuzzy* degli archivi catastali storici (dal 1830 ad oggi).

Motivazione della scelta del tema e dell'approccio

La maggior parte dei siti web aziendali per software house si affida a template generici privi di identità semantica, oppure a CMS complessi che appesantiscono le prestazioni e frammentano il controllo sul codice. La scelta di creare un portale ad hoc per Algora Studio risponde alla necessità di dimostrare come la programmazione web nativa (**HTML5, CSS3, JavaScript Vanilla, PHP e MySQL**) consenta di realizzare un'applicazione web con:

1. Un'estetica altamente caratterizzata ("Archivio Caldo") coerente con il dominio trattato.
 2. Prestazioni e tempi di caricamento ridotti al minimo (stylesheet unico di ~12 KB, zero dipendenze esterne).
 3. Piena accessibilità e aderenza totale ai vincoli d'esame e di validazione W3C.
-

2. ARCHITETTURA DELL'INFORMAZIONE E MAPPATURA DELLE PAGINE

L'applicazione web si articola su **5 pagine HTML5 statiche principali** integrate da **1 script server-side dinamico PHP** per la gestione della persistenza dei dati:

RISORSA FILE	STACK TECNOLOGICO	METODOLOGIA, CONTENUTO E RUOLO NELL'ARCHITETTURA
<code>index.html</code>	HTML5, CSS3, JS	Home Page Istituzionale: Definisce la vision di Algora Studio, presenta il manifesto aziendale ("Strumenti digitali per chi custodisce la conoscenza"), evidenzia le metriche chiave e fornisce un teaser del caso studio reale.
<code>chi-siamo.html</code>	HTML5, CSS3	Profilo Aziendale e Filosofia: Illustra la filosofia dello "sviluppo verticale", i 3 valori guida (Specializzazione, Co-progettazione, Tecnologie durevoli) e presenta il profilo del fondatore e lo stack tecnologico adottato.
<code>foliarium.html</code>	HTML5, CSS3	Scheda Prodotto Verticale: Dettaglia le funzionalità del software <i>Foliarium</i> (ricerca fuzzy Levenshtein, albero delle proprietà, audit trail, esportazione report PDF/Excel), i requisiti di sistema e la struttura dei prezzi/licenze per la Pubblica Amministrazione.
<code>caso-studio.html</code>	HTML5, CSS3	Caso Studio Archivio di Stato di Savona: Documenta i risultati quantitativi (69 comuni, 12.000+ partite catastali, 8.500+ possessori) e confronta il processo di ricerca prima e dopo la digitalizzazione tramite un layout comparativo.

RISORSA FILE	STACK TECNOLOGICO	METODOLOGIA, CONTENUTO E RUOLO NELL'ARCHITETTURA
<code>contatti.html</code>	HTML5, CSS3, JS	Interfaccia di Contatto e Demo: Combina una guida conoscitiva in 4 fasi, un sistema di FAQ interattivo ad accordion e il modulo per la prenotazione della demo gratuita.
<code>invia-contatto.php</code>	PHP 8.x, MySQLi	Backend Server-Side: Riceve le richieste HTTP POST dal form di contatto, applica sanitizzazione/validazione, memorizza i dati nel database MySQL tramite Prepared Statements e genera un esito HTML dinamico.

3. TECNICHE FRONTEND: HTML5 SEMANTICO, CSS3 E DESIGN SYSTEM

3.1 Scrittura Nativa e Struttura Semantica

In linea con i requisiti del corso, il codice è stato scritto interamente a mano senza ricorrere a editor WYSIWYG o framework CSS pesanti (come Bootstrap o Tailwind). La scelta di non utilizzare framework garantisce il pieno controllo sull'albero DOM e sull'ingombro del codice stylesheet (ridotto a un singolo file `style.css` di ~12 KB).

Ogni pagina utilizza tag semantici HTML5 per strutturare i contenuti: `<nav>` per la navigazione globale, `<header>` per le sezioni introduttive, `<main>` e `<section>` per i blocchi tematici principali, `<footer>` per la chiusura di pagina.

3.2 Design System "Archivio Caldo" e Tipografia Modulare

Per trasmettere il senso di storicità, autorevolezza e precisione archivistica, è stato definito un Design System basato su CSS Custom Properties (Variabili CSS):

- **Palette Cromatica:** Sfondo pergamena tenue (`--parchment: #F4EFE4`, `--cream: #F9F6EF`), testo e contenitori inchiostro profondo (`--ink: #1E150A`), dettagli e accenti dorati (`--gold: #B8821A`) e verde archivio per le azioni confermate (`--green: #1A3A28`).
- **Accoppiamento Tipografico:** Utilizzo dei font Google Fonts *Cormorant*

Garamond (serif elegante per i titoli display `h1`, `h2`, `h3`) e *Libre Baskerville* (serif ad alta leggibilità per il corpo del testo). I micro-testi e le etichette tecniche utilizzano *Trebuchet MS / Calibri* in maiuscolo con spaziatura tracciata (`letter-spacing: 3px-5px`).

3.3 Layout Non Lineare: CSS Grid, Flexbox e Posizionamento Ancorato

Il requisito del layout "non lineare" è stato soddisfatto integrando diverse tecniche di posizionamento al di fuori del normale flusso del documento:

1. **Header Fisso (Fixed Positioning):** La barra di navigazione principale (`#nav`) ha un posizionamento ancorato `position: fixed; top: 0; left: 0; right: 0; z-index: 200;` che la mantiene costantemente visibile durante lo scroll della pagina.
 2. **Griglie Asimmetriche CSS Grid:** La hero section e la struttura del caso studio utilizzano griglie a colonne differenziate (es. `grid-template-columns: 1fr 400px;` o `repeat(3, 1fr)` con gap di 2px su sfondo contorno per creare l'effetto griglia "a caselle archivistiche").
 3. **Confronto "Prima e Dopo" (Before/After):** In `caso-studio.html` è stato realizzato un layout a due colonne asimmetriche con sfondi a contrasto (parchment vs ink) per evidenziare visivamente i vantaggi della digitalizzazione.
-

4. TECNICHE CLIENT-SIDE: JAVASCRIPT VANILLA E INTERATTIVITÀ

I moduli client-side sono stati implementati in JavaScript moderno (ES6+) Vanilla, con l'obiettivo è stato quello di arricchire l'esperienza utente mantenendo tempi di esecuzione immediati.

- **Menu Hamburger Responsive (`toggleNav`):** Permette l'apertura e la chiusura del menu di navigazione su schermi mobili. Al container `div.nav-burger` sono stati aggiunti gli attributi di accessibilità W3C validati `role="button"`, `tabindex="0"` e `aria-label="Menu"` per la navigazione via tastiera.
- **Scroll Event Listener dinamico:** Un event listener ascolta lo scorrimento della finestra (`window.addEventListener('scroll', ...)`) e aggiunge la classe

`.scrolled` al navigatore superati i 20px, attivando un'ombra sfumata che stacca la barra dal contenuto sottostante.

- **FAQ Accordion Interattivo (`toggleFaq`):** In `contatti.html`, un sistema a fisarmonica gestisce la chiusura automatica delle altre risposte al click su una nuova domanda, calcolando dinamicamente la proprietà CSS `max-height` e la rotazione dell'icona `+`.
-

5. TECNICHE SERVER-SIDE E PERSISTENZA DATI: PHP & DBMS MYSQL

5.1 Motivazione dell'Architettura Server-Side

Un semplice form HTML che invia un'email diretta via `mailto:` o che simula l'invio via JS non è sufficiente per garantire la persistenza dei dati, il tracciamento delle richieste e la sicurezza. Si è scelto pertanto di implementare un'architettura 3-Tier standard: **Client (HTML/CSS/JS) → Application Server (PHP 8) → Database Server (MySQL/MariaDB)**.

5.2 Progettazione del Database (`algora_db`)

Il database relazionale locale memorizza le richieste di contatto e prenotazione demo arrivate dal sito. Lo schema DDL è stato progettato con tipi di dato ottimizzati e vincoli di integrità:

```
-- Creazione e selezione del Database
CREATE DATABASE IF NOT EXISTS algora_db CHARACTER SET utf8mb4
COLLATE utf8mb4_unicode_ci;
USE algora_db;

-- Creazione della tabella per i contatti dal form web
CREATE TABLE IF NOT EXISTS contatti (
    id INT AUTO_INCREMENT PRIMARY KEY,
    nome VARCHAR(100) NOT NULL,
    ente VARCHAR(100),
    email VARCHAR(100) NOT NULL,
    tipo VARCHAR(50),
    messaggio TEXT NOT NULL,
```

```
data_invio DATETIME DEFAULT CURRENT_TIMESTAMP
) ENGINE=InnoDB;
```

5.3 Flusso di Elaborazione e Sicurezza Backend (invia-contatto.php)

All'invio del modulo, il browser invia una richiesta HTTP `POST` a `invia-contatto.php`. Lo script esegue i seguenti passaggi in sequenza rigorosa:

- 1. Verifica del Metodo HTTP:** Controlla che la richiesta sia arrivata via `$_SERVER["REQUEST_METHOD"] === "POST"` per impedire l'accesso diretto via URL.
- 2. Sanitizzazione e Validazione Severa:**
 - `trim()` e `htmlspecialchars()` per rimuovere spazi bianchi eccedenti e convertire i caratteri speciali HTML (es. `<`, `>`, `&`), neutralizzando attacchi di tipo *Cross-Site Scripting (XSS)*.
 - `filter_var($email, FILTER_VALIDATE_EMAIL)` per verificare la correttezza formale dell'indirizzo di posta elettronica lato server.
- 3. Istruzioni Prepare (Anti-SQL Injection):** L'inserimento nel database avviene tramite Prepared Statements della libreria `mysqli`:

```
$stmt = $conn->prepare("INSERT INTO contatti (nome, ente, email, tipo, messaggio) VALUES (?, ?, ?, ?, ?)");
$stmt->bind_param("sssss", $nome, $ente, $email, $tipo, $messaggio);
$stmt->execute();
```

L'uso dei segnaposto `?` e il binding separato dei parametri separano completamente le istruzioni SQL dai dati forniti dall'utente, rendendo impossibile qualsiasi attacco di *SQL Injection*.

- 4. Generazione dell'Output HTML Dinamico:** In base all'esito dell'operazione, lo script PHP renderizza direttamente una pagina di conferma personalizzata (con il nome dell'utente e l'email inserita) o una scheda d'errore stilizzata con lo stesso layout e foglio di stile `style.css` del sito.
-

6. USABILITÀ, ACCESSIBILITÀ E RISOLUZIONE ERRORI W3C

Durante la fase di collaudo, tutte le pagine sono state sottoposte alla verifica formale tramite il **W3C Markup Validation Service**. Sono state individuate e risolte le seguenti criticità:

SEGNALAZIONE W3C	CAUSA TECNICA	SOLUZIONE IMPLEMENTATA
Error: aria-label on div	L'attributo <code>aria-label</code> era stato inserito direttamente su un <code>div</code> generico senza ruolo ARIA.	Aggiunti <code>role="button"</code> e <code>tabindex="0"</code> all'elemento <code>div.nav-burger</code> per renderlo semanticamente un pulsante accessibile anche da tastiera.
Error: Skipping heading level	Presenza di tag <code>h4</code> direttamente sotto sezioni gestite con <code>h2</code> (es. box stack e colonne footer).	Ristrutturata l'alberatura dei titoli sostituendo gli <code>h4</code> con <code>h3</code> sia nei file HTML che nelle regole CSS del footer e della sidebar.
CSS Inline warnings	Utilizzo temporaneo di attributi <code>style="..."</code> all'interno dei div di layout.	Centralizzati i blocchi di stile all'interno del file unico <code>style.css</code> per mantenere la massima pulizia del codice.

7. ISTRUZIONI PER L'INSTALLAZIONE LOCALE

Per eseguire il collaudo completo della piattaforma e verificare l'interazione dinamica con il database relazionale:

1. Avviare i moduli **Apache** e **MySQL** all'interno del proprio ambiente locale (XAMPP, MAMP o WAMP).
2. Copiare l'intera cartella `algora_site` all'interno della directory root del server (es. `C:\xampp\htdocs\algora_site` oppure nella cartella `C:\wamp64\www\algora_site` nel caso di WAMP).
3. Aprire **phpMyAdmin** (<http://localhost/phpmyadmin>) ed eseguire le query DDL di creazione del database `algora_db` e della tabella `contatti` riportate al punto 5.2.
4. Aprire il browser e digitare l'URL locale per la home: http://localhost/algora_site/index.html

5. Per testare le funzionalità server basta andare su `http://localhost/algora_site/contatti.html`, compilare il form e inviare i dati. Verificare che venga mostrata la schermata di conferma dinamica generata da `invia-contatto.php` e che la nuova riga sia presente nella tabella MySQL

8. ISTRUZIONI PER L'UTILIZZO ONLINE

1. In alternativa la versione online è disponibile per la consultazione al sito www.algorastudio.it.
2. Nel caso di utilizzo su hosting e dominio web, bisogna configurare nel file .php i parametri di connessione del database e del relativo user con i privilegi associati, impostati sul gestore del dominio.
3. Qui una schermata di **phpMyAdmin** che dimostra la corretta creazione del record relativa alla richiesta di contatto.

✓ Mostro le righe 0 - 1 (2 del totale, La query ha impiegato 0.0002 secondi.)

SELECT * FROM `contatti`

☐ Profiling [Modifica inline] [Modifica] [Spiega SQL] [Crea il codice PHP] [Aggiorna]

☐ Mostra tutti | Numero di righe: 25 ▼ | Filtra righe: Cerca nella tabella | Ordina per chiave: Nessuno ▼

Opzioni extra

	id	nome	ente	email	tipo	messaggio	data_invio
☐ Modifica ☐ Copia ☐ Elimina	3	Marco Santoro	Savona	santoromarco@gmail.com	Università / Centro di ricerca	Facciamo una prova	2026-08-19 18:53:36

☐ Seleziona tutto | Se selezionati: ☐ Modifica ☐ Copia ☐ Elimina ☐ Esporta

☐ Mostra tutti | Numero di righe: 25 ▼ | Filtra righe: Cerca nella tabella | Ordina per chiave: Nessuno ▼